

Day 23: Modern C++ in Embedded (C++11/14/17)

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

constexpr Functions and Variables

`constexpr` (C++11) tells the compiler: evaluate this at **compile time**. The result is embedded in the binary as a constant — zero runtime computation:

```
constexpr uint32_t CPU_FREQ_HZ = 64'000'000UL;           // C++14 digit separator
constexpr uint32_t TICK_NS      = 1'000'000'000 / CPU_FREQ_HZ;

// constexpr function: can be called at compile OR runtime
constexpr uint16_t adc_to_mv(uint16_t raw, uint16_t vref = 3300) {
    return (uint32_t)raw * vref / 4095;
}
static_assert(adc_to_mv(2048) == 1650, "Mid-scale should be 1650mV");
```

nullptr vs NULL

`nullptr` (C++11) is type-safe — it has type `std::nullptr_t` and cannot accidentally match integer overloads:

```
void func(int);           // Overload 1
void func(void *);        // Overload 2
func(NULL);               // Ambiguous! NULL is often (void*)0 or 0 (int)
func(nullptr);            // Unambiguous: calls void func(void *)
```

Always use `nullptr` in modern C++ embedded code.

noexcept Specifier

`noexcept` tells the compiler (and callers) that a function **will not throw exceptions**. Since embedded disables exceptions anyway, marking hot functions `noexcept` enables additional compiler optimizations:

```
static inline bool gpio_toggle(int pin) noexcept {
    return gpio_pin_toggle(gpio_dev, pin) == 0;
}
```

Scoped enum class

`enum class` (C++11) creates a **scoped, strongly-typed enum** — values don't implicitly convert to `int` and don't pollute the enclosing namespace:

```
enum class BaudRate : uint32_t {
    Baud9600    = 9'600,
    Baud115200 = 115'200,
};
BaudRate b = BaudRate::Baud115200;
uint32_t raw = static_cast<uint32_t>(b); // Must cast explicitly
```

Uniform Initialization {}

Brace initialization (C++11) works for all types and prevents narrowing conversions:

```
uint8_t a{255}; // OK
uint8_t b{256}; // COMPILE ERROR: narrowing (256 doesn't fit in uint8_t)
std::array<uint8_t, 4> buf{0xAA, 0xBB, 0xCC, 0xDD}; // Aggregate init
```

auto Keyword

`auto` deduces the type from the initializer — reduces verbosity for complex types:

```
auto *dev = DEVICE_DT_GET(DT_NODELABEL(uart0)); // const struct device *
auto result = read_adc(0);                      // Result<uint16_t>
```

Don't overuse `auto` — it can hide types and make embedded code harder to reason about. Prefer explicit types for register values and bit fields.

if constexpr (C++17)

`if constexpr` evaluates the condition at **compile time** — the false branch is not compiled:

```
template <typename T>
void process(T val) {
    if constexpr (sizeof(T) == 1) {
        // This code only compiles when T is 1 byte
        uart_poll_out(dev, val);
    } else if constexpr (sizeof(T) == 4) {
        // This code only compiles when T is 4 bytes
        send_u32(val);
    }
}
```

Replaces `#ifdef` for template branching — much safer and cleaner.

Structured Bindings (C++17)

Unpack structs, pairs, and arrays into named variables:

```
auto [pin, active_low, debounce_ms] = get_button_config();
auto [value, error_code]           = read_adc_result();
```

Practice Tasks

1. Create a `constexpr` lookup table for a CRC-8 polynomial — verify it's computed at compile time (check disassembly)
2. Write a template function that uses `if constexpr` to handle 8-bit and 32-bit register sizes differently
3. Refactor a raw `enum` to `enum class` in your state machine code — fix all the required `static_cast` callsites
4. Use structured bindings to unpack your `Result` type — compare readability vs `.value` / `.err` access

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day23
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day23/main.cpp](#)